Assignment 1 : HTML and CSS

Overview

This assignment has a written part (Part A) and three build questions. Question 1 is your personal portfolio website, which you will keep working on as you learn new topics. Questions 2 and 3 are separate tasks for imaginary clients.

Be creative. The elements listed in each question are the minimum. Use your own layout, colours, fonts, images and wording so that your work looks different from everyone else's.

Part A: Written questions

Answer in a file named `answers.txt`. Keep each answer short and in your own words.

1. A user opens your portfolio from a link. Describe step by step what happens between the browser and the server, and draw a simple client–server diagram (hand-drawn and photographed is fine). Say where the app logic and the database sit.
2. Your contact form sends a visitor's message to a server. Should it use GET or POST, and why? What happens if `method` is left out? Name the status code the server returns on success and the family of codes that means the server failed.
3. Name the three types of CSS and say where each is written. If all three colour the same paragraph differently, which wins? Which type would you use for a style shared by four pages, and why?
4. A page has `<p id="intro" class="note">Hello</p>`. Rules exist for `*`, `p`, `.note` and `#intro`, each with a different `color`. Which applies, and what is the full order of precedence from strongest to weakest?
5. Explain `padding` and `margin` with a small drawing of a `<div>` showing content, padding, border and margin. What does `box-sizing: border-box` change?
6. Where does JavaScript code run, and what is the JS engine in a browser? Why is no compiler needed to run JavaScript? Name two things JavaScript can do on a web page that HTML and CSS alone cannot.
7. JavaScript has no type safety. Explain what happens when you write `a = 5;` and then `a = "hello";`. Name the three data types you have learned so far.
8. Explain the four values of the CSS `position` property (`static`, `relative`, `absolute`, `fixed`). When would you use `top`, `left`, `right` and `bottom`?

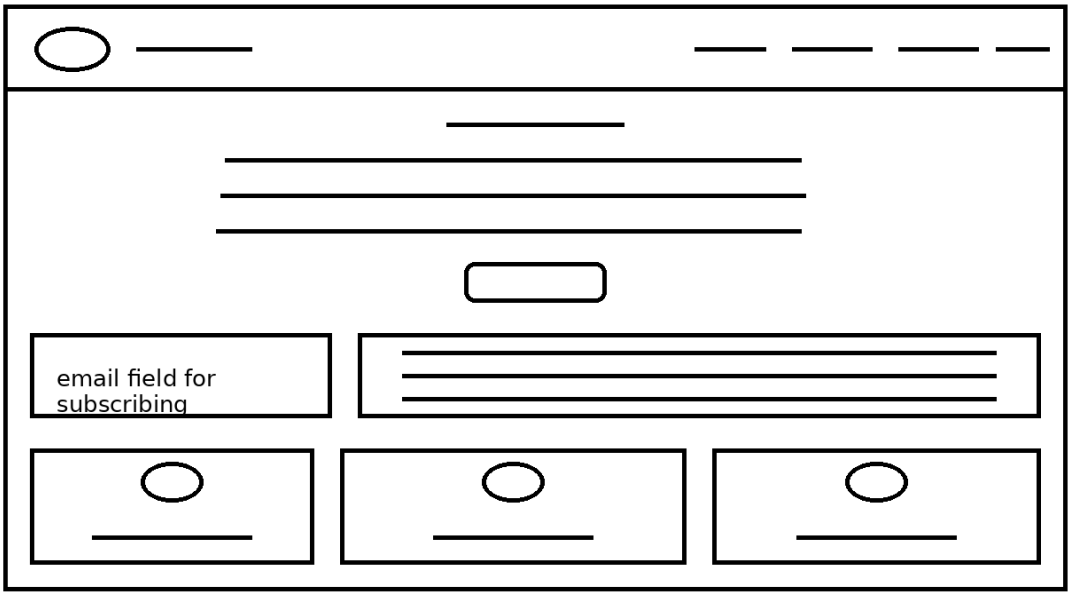
9. A container has `display: flex` and three boxes with `flex` values of 1, 2 and 1. How is the width shared between the boxes? What do `gap` and `justify-content: center` do?

Question 1: Your portfolio website

Brief. Recruiters look at a developer's portfolio before they read the CV. Build a three-page website about yourself using real content. Choose your own theme, colour palette, tagline and layout. You will keep working on this project as you learn new topics, so keep your code tidy.

Home page layout

Design your Home page using the same layout that was sketched in class. Colours, fonts, images and wording are yours to choose, but the structure must match.



1. **Header.** A logo image and your site name on the left, and four navigation links on the right (Home, Projects, Contact and one more of your choice). The header stays fixed at the top.
2. **Hero section.** A centred heading, a short paragraph about you, and a button such as "View my projects" that links to the Projects page.
3. **Middle row.** On the left, a small box with an email field and a Subscribe button for visitors who want updates. On the right, a wide box with your "About me" text.

4. **Card row.** Three cards side by side, each with an image or icon above a title. Use them for Education, Skills and Projects.

Build the header, the middle row and the card row with Flexbox. The Home page requirements below still apply: put your education list and skills list inside the cards.

Home page (`index.html`)

- A page `<title>`, an `<h1>` with your name, and at least one `<h2>` and `<h3>`.
- A short "About me" in `<p>`, with your role or tagline highlighted using `<span>`.
- A profile photo using `<img>` with `width` and `height`, stored in `images/` and referenced with a correct relative path.
- An **ordered list** for your education timeline (at least 3 entries).
- An **unordered list** for your skills (at least 5 items).
- At least two links: one external (GitHub, LinkedIn) and one to another page of your site.

Projects page (`projects.html`)

- A **table** of at least 4 projects with columns **Title, Tech used, Description, Link**. Use `<thead>`, `<tbody>` and `<th>`. If you have fewer than 4 real projects, use course exercises or practice projects and label them honestly.
- Below the table, show your top two projects as **cards**. Each card is a `<div>` with a heading, a short paragraph and a link.

Contact page (`contact.html`)

- A `<form>` with an `action` (dummy URL) and the `method` you justified in Part A.
- Fields: name (text), email (email), preferred contact date (date), enquiry type (`select` with at least 3 options), preferred contact mode (radio buttons sharing one `name`), message (`textarea`), CV upload (file), and a consent checkbox.
- Every field has a `<label>`, a `placeholder` where it makes sense, and both an `id` and a `name`.
- Submit and Reset buttons.
- An `<iframe>` embedding a map of your city or college, or a video you made.

CSS across the site

1. **External CSS.** One `style.css` linked from all three pages. It styles the `body`, headings, navigation, table borders and header cells, form labels and the submit button.

2. **Selectors.** Tag selectors for the basics. An id selector for one unique element, for example `#main-nav`. A class selector, such as `.card`, used on at least three elements. A universal `*` rule. A comment above them stating the order of precedence.
3. **Internal and inline CSS.** One page-specific `<style>` block on one page, and one `style` attribute on one element. A comment on each explains why you chose that type.
4. **Box model.** Cards use `width`, `border`, `border-radius`, `padding` and `margin`.
5. **Flexbox layout.** Build the navigation row and the row of project cards with `display: flex`. Use `gap` and `justify-content`, and give at least two cards different `flex` values.
6. **Position.** Keep the navigation bar at the top of the window while scrolling with `position: fixed` and `top`, `left` and `right`. Use `position: relative` on a card and `position: absolute` on a small label inside it, for example a "New" badge. A comment explains how `static`, `relative`, `absolute` and `fixed` differ.
7. **Hover and gradient.** Navigation links and buttons change colour when the mouse is over them, with a smooth `transition`. Use `linear-gradient` for at least one background.

Preparing for JavaScript

You will add JavaScript to this site in a few weeks, so write your code in a way that makes this easy.

- Give every important element a meaningful `id` or `class`, such as `skills-list`, `project-table`, `contact-form`. No `div1`, `box2`.
- Keep related content together and consistent, for example every project row has the same columns in the same order.
- Add comments marking places where you think something could become interactive, such as `<!-- later: dark mode button here -->`.
- Create an empty `js/` folder in your project now.

Question 2: HR portal (login, directory and onboarding form)

Brief. TechNova Solutions is replacing its paper joining process. HR wants three pages: a login page, a directory of current employees and a form for new joiners. Build all three as a client deliverable, in a folder separate from your portfolio.

Directory page (`directory.html`)

- A table of at least 6 employees with columns **Emp ID, Name, Department, Email, Phone**. Use `<thead>`, `<tbody>` and `<th>`, with `border`, `cellpadding` and `cellspacing` set so it reads well.
- A line below the table stating the total number of employees.
- Use dummy data only. No real people's details.

Onboarding page (`onboarding.html`)

Field	Required input
Full name	text, with placeholder
Official email	email
Temporary password	password
Date of birth	date
Joining date	date
Orientation slot	datetime-local
Reporting time on day one	time
Gender	radio buttons sharing one `name`
Highest qualification	dropdown with at least 4 options, each with a `value`
Department	dropdown with at least 4 options
Current address	textarea
Passport photo or resume	file
Declaration	checkbox

- Wrap everything in a `<form>` with an `action` and a justified `method`. Every field has a `<label>`.
- Include Submit and Reset buttons, plus one plain button (for example "Need help?") with a comment explaining why it does nothing yet.
- Below the form, embed a map of the TechNova office with an `<iframe>`.

Login page (`login.html`)

- A login card made with a `<div>`: fixed `width` and `height`, white background, rounded corners, `text-align: center`, and centred on the page with `margin: auto`.
- A page background that uses `linear-gradient`.
- A heading, a username input, a password input and a Login button. Below them, a line such as "Need an account? Sign Up" with a link that has no underline.
- Style the inputs with a class selector: only a bottom border, no outline, and comfortable `padding`.
- Style the button using an id selector. It changes colour when the mouse is over it, with a smooth `transition`.
- Choose your own colours and fonts. Do not copy the example shown in class.

Shared styling

All three pages link one external stylesheet and have a navigation row to move between them.

Question 3: Landing page for a local business

Brief. A local business of your choice (a café, gym, bakery, tuition class) wants a one-page website that shows what it offers and its prices. Invent the business and its content. Pick an unusual idea and let the look of the page match the business. This question focuses on CSS and the box model.

Build a single page, `landing.html`, with its own `landing.css`.

- **Content.** A banner image, an `<h1>`, a short intro in `<p>` with a `<span>` highlight, an ordered list of "How to order or join" steps, an unordered list of opening hours, and one external link.
- **Service cards.** At least three offers (menu items, plans, batches), each as a `<div>` card with a heading, a paragraph and a price. Each card sets `width`, `height`, `background-color`, `color`, `border` (`border-width`, `border-style`, `border-color`), `border-radius`, `padding`, `margin` and `box-sizing: border-box`. Comment which spacing is inside the border and which is outside.
- **Selectors.** Tag selectors for the basics, an id selector for the banner or header, a class selector shared by all cards and by at least one other kind of element, and a universal `*` rule. A comment states the order of precedence.
- **Text properties.** Use at least four of `font-size`, `font-family`, `font-weight`, `text-transform`, `text-decoration`, each for a clear purpose.

- **Colours.** Use all three ways to write a colour: a name, a hexadecimal value such as `#3b5998`, and `rgb(r, g, b)`.
- **Three types of CSS.** `landing.css` is the external sheet. Add a short internal `<style>` rule and one inline `style` attribute, with a comment saying which wins when they clash.
- **Flexbox layout.** Place the cards side by side in a container with `display: flex`, `gap` and `justify-content: center`. Give the featured card a larger `flex` value than the others.
- **Position and hover.** Add an "Order now" or "Call us" button fixed to the bottom-right corner of the window with `position: fixed`. Cards and buttons change colour on hover with a `transition`. Use `linear-gradient` for at least one background.

Folder structure and Rules

Organise the work exactly like this. Broken image or stylesheet paths count as errors.

```
<YourName>_Assignment1/
├── answers.txt
├── client-server-diagram.png
├── portfolio/
│   ├── index.html
│   ├── projects.html
│   ├── contact.html
│   ├── style.css
│   ├── images/
│   └── js/          (empty for now)
├── hr-portal/
│   ├── login.html
│   ├── directory.html
│   ├── onboarding.html
│   └── style.css
└── landing-page/
    ├── landing.html
    ├── landing.css
    └── images/
```

Rules

- Use only HTML and CSS. No JavaScript, no frameworks, no CSS libraries, no templates downloaded from the internet.
- Write your own content.
- Indent consistently and comment decisions that need explaining.
- Use semantic tags (`<thead>`, `<tbody>`) wherever they apply.

- Test every link, image, stylesheet and form button in a browser before submitting.
- Keep the portfolio in its own folder. You will keep working in it after this assignment.

Bonus (optional)

Attempt these only after finishing everything above.

- Add a fourth portfolio page, `blog.html` or `certificates.html`, linked from the navigation.
- Group the onboarding form with `<fieldset>` and `<legend>` into "Personal", "Education" and "Job details".
- Research `flex-wrap` and `align-items`. Use them so the cards drop to a second line when the window is narrow and sit in the vertical centre of their row. This goes beyond what has been taught so far.

Note: After completion, send the screenshots/videos of your work on the PGCP-AC Group.